

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:100

Annexure A

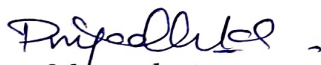
Debugging or Optimization task

Code of Conduct:

192511248

I Priyadharshini K (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.


Signature of the student:

Q1. Deadlock Debugging (Code Trace)

You are given the following pseudo-code for two processes sharing resources R1 and R2:

Process P1:
lock(R1)
wait(100 ms)
lock(R2)
critical section
unlock(R2)
unlock(R1)

Process P2:
lock(R2)
wait(100 ms)
lock(R1)
critical section
unlock(R1)
unlock(R2)

Identify the exact condition that causes deadlock.

Suggest an optimisation to eliminate the deadlock without changing process logic.

(10 Marks)

Q2. CPU Utilisation Problem

A system shows low CPU utilisation (30%) even though processes are ready.

You analyse the scheduler log and find frequent transitions:

ready → waiting → ready → waiting → ready ...

Debug the cause using process management concepts and propose a system-level optimisation to improve utilisation.

(10 Marks)

Q3. Memory Bottleneck Debugging

A multi-threaded program frequently encounters page faults, slowing performance. The memory map shows:

- Code: 5 MB

- Heap: 150 MB
- Stack: 1 MB per thread
- 25 threads running
- Physical RAM available: 2 GB

Identify the memory issue causing performance degradation and propose two optimisations.

(10 Marks)

Q4. Starvation Debugging

You observe that process P5 never gets CPU time in a multilevel queue scheduler:

- Q1 (Highest Priority): Interactive
- Q2: System
- Q3: Batch
- Time Quantum: Q1 = 5ms, Q2 = 10ms, Q3 = FCFS

P5 is in Q3 but keeps getting postponed.

Debug the root cause of starvation and suggest an optimisation to ensure fairness.

(10 Marks)

Q5. Context-Switch Overhead Debugging

A server performs 20,000 context switches/sec, causing high overhead.

Log details show:

- Average burst time per thread: 2ms
- I/O wait frequency: very high
- Preemptive scheduler with quantum = 1ms

Explain why context switching is so high and propose two optimisations to reduce overhead.

(10 Marks)

Q6. Producer–Consumer Debugging (Semaphore Error)

A producer-consumer application occasionally hangs after running for several minutes.

Semaphore empty = 5
Semaphore full = 0
Semaphore mutex = 1

Producer:

```
wait(mutex)
wait(empty)
insert(item)
signal(full)
signal(mutex)
```

Consumer:

```
wait(mutex)
wait(full)
remove(item)
signal(empty)
signal(mutex)
```

The program freezes even though both producer and consumer threads are alive.

1. Identify the synchronization error causing the hang.
2. Explain why the semaphore ordering is incorrect.
3. Suggest the correct semaphore sequence to eliminate the problem.

(10 Marks)

Q7. Thread Performance Debugging

A web server creates one new thread for every incoming client request.

Performance statistics:

- Average requests/sec = 15,000
- Average thread creation time = 0.4 ms
- CPU utilisation = 92%
- Memory usage continuously increases

Developers notice that the server becomes slow after several hours.

1. Identify the thread-management problem.
2. Explain why the current multithreading model causes performance degradation.
3. Propose two optimisations to improve scalability.

(10 Marks)

8. Banker's Algorithm Debugging

Available Resources:

A B C = 2 1 1

Allocation:

P1: 1 0 1

P2: 0 1 0

P3: 1 1 1

Maximum:

P1: 2 1 2

P2: 1 2 1

P3: 3 2 2

Process P2 requests (1,1,1).

1. Calculate the Need matrix.
2. Determine whether the request can be granted immediately using Banker's Algorithm.
3. If not, identify the unsafe condition and explain why granting the request could lead to deadlock.

(10 Marks)

Q9. Deadlock Detection and Recovery Debugging

A database server reports that four transactions are waiting for resources.

Resource Allocation Graph:

- T1 waits for R2
- T2 waits for R3
- T3 waits for R4
- T4 waits for R1
- R1 allocated to T1
- R2 allocated to T2
- R3 allocated to T3
- R4 allocated to T4

The database becomes completely unresponsive.

Identify whether a deadlock exists.

Explain how deadlock detection identifies this situation.

Suggest two recovery techniques and justify which one minimises system disruption.

(10 Marks)

Q10. Monitor-Based Synchronization Debugging

A monitor is used to synchronize access to a shared printer.

monitor Printer

```
{  
    boolean busy = false;
```

```
    procedure Print()  
    {
```

```
        if(busy)  
            wait();
```

```
        busy = true;
```

```
        // Printing
```

```
        busy = false;
```

```
    }  
}
```

Users report that some print jobs never execute even though the printer becomes free.

1. Identify the logical error in the monitor implementation.
2. Explain why waiting processes are never awakened.
3. Modify the monitor logic to ensure correct synchronization and fairness.

(10 Marks)

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Question No.	Problem Identification (20%)	Debugging(25%)	Optimization (20%)	Analysis (20%)	Code Quality (15%)	Total Marks (100)
Q1						
Q2						
Q3						
Q4						
Q5						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

NAME: PRIYADHARSHINI K'S

REG: 192511248

SUB: OPERATING SYSTEMS

CODE: CSA 0404

CO2 AT3 DEBUGGING

Deadlock Debugging [code Trace]

(i) Identify exact condition that causes deadlock

A deadlock occurs because both processes acquire the resources in a different order.

Process P1:

Locks R1

Waits for 100ms

Tries to lock R2

Process P2:

Locks R2

Waits for 100ms

Tries to lock R1

If P1 locks R1 and P2 locks R2 at same time, then:

P1 waits for R2, which is held by P2

P2 waits for R1, which is held by P1

Both process wait forever, resulting in a deadlock.

Deadlock condition present :

- 1) Mutual exclusion
- 2) Hold and wait
- 3) No Preemption
- 4) Circular wait

(ii) Suggest an optimisation to eliminate deadlock without changing process logic.

Optimization: Use a fixed resource ordering

Both process should acquire resources in same order:

Process P1:

lock (R1)

lock (R2)

Critical section

unlock (R2)

unlock (R1)

Process P2:

lock (R1)

lock (R2)

Critical section

unlock (R2)

unlock (R1)

Since both processes request R_1 first and R_2 second, the circular wait condition is removed, preventing deadlock.

2) CPU utilization problem:

↳ Debug cause using process management:

CPU is utilized only 30% even though processes are ready because the processes are frequently switching between Ready to waiting states.

Reason:

- * Process perform I/O operations very frequently.

- * They move from Ready $\rightarrow$ Waiting to wait for I/O completion.

- * After I/O completes, they return to Ready state.

- * This repeated switching leaves CPU idle at times, resulting in low CPU utilization.

- * This is known as I/O-bound workload where processes spend more time waiting for I/O than executing on CPU.

9th System - level optimization!

01 → Improve CPU scheduling

Use multilevel Feedback Queue [MLFQ] or Shortest Remaining Time First [SRTF] scheduling.

These algorithms prioritize ready processes and reduce CPU idle time.

02 → Reduce I/O waiting

Use asynchronous I/O, buffering, or caching so that I/O operations overlap with CPU execution.

This allow other ready processes to use the CPU while one process waits for I/O.

3) Memory Bottleneck Debugging:

↳ memory issue:

The main issue is excessive page faults caused by the multi-threaded application.

Given,

Code = 5MB

Heap = 150mb

Stack = 1mb per thread

No. of threads = 25

Total Stack memory = 25mb

Physical RAM = 2Gb

Although 2GB of RAM is available, the large heap, multiple thread stacks, and poor memory access patterns can lead to frequent page faults.

CPU spends more time loading pages from disk into memory instead of executing instructions, which slows down program.

(ii) Optimisations :

1 → Reduce memory usage

* Decrease number of threads or reduce stack size of each thread.

* This lowers memory consumption & reduces page faults.

2 → Improve memory management

Use efficiency memory allocation &
Improve data locality

This increases cache efficiency &
reduces page faults.

4) Starvation Debugging:

(i) Root cause:

In the given multilevel queue scheduler:

Q1 [Highest priority]: Interactive [5ms]

Q2 : System (10ms)

Q3 : Batch (FCFS)

Q3 contains P5.

Root cause is that Q1 & Q2 are continuously receiving process. Since the scheduler always serves higher priority queues P5 in Q3 will never gets CPU time.

(ii) Optimization: Ageing

Gradually increase priority of process that has been waiting for a long time.

P5 is promoted to a higher - priority queue & gets CPU time.

Optimization 2: Fair CPU allocation

Allocate a small percentage of CPU time to lower-priority queues even when higher-priority queues are busy.

This ensures that all processes receive CPU time & prevents starvation.

5) Context - Switch Overhead:

(i) Given,

Context switches = 20,000 per sec.

Average burst time = 2ms

I/O wait frequency = Very high

Time quantum = 1ms

The context-switch is high because:

Time quantum (1ms) is smaller than average CPU burst time (2ms), so processes are preempted before completing.

Frequent I/O operations cause processes to move repeatedly between Running & Waiting states.

→ Improve memory management.

ii) Optimization 1 → Increase Time Quantum

Increase time quantum from 1ms to a larger value [Eg. 4-5ms]

This reduces unnecessary preemptions and lower number of context switches.

Optimization 2 → Improve I/O Handling

Use asynchronous I/O, buffering, or thread pooling to reduce I/O wait frequency.

This decreases process blocking & minimizes context switches.

b) Producer-Consumer [Semaphore error]

iii) The synchronization error is the incorrect order of semaphore operations.

Both producer & consumer lock the mutex before checking empty or full Semaphore. If required semaphore is unavailable, the process blocks while still holding mutex.

(i)

Current order is incorrect because

A process may block on wait (empty) or wait (full) while holding the mutex.

Since mutex is locked, other process can't enter critical section to update.

This results in deadlock [hang].

(ii) Correct Semaphore:

Producer:

wait (empty)

wait (mutex)

insert (item)

signal (mutex)

signal (full)

Consumer:

wait (full)

wait (mutex)

remove (item)

signal (mutex)

signal (empty)

In this, availability of buffer space (empty) or data (full) is checked before locking mutex.

1) Thread performance :

i) Problem : A web server creates a new thread for every incoming client request.

Since server handles 15,000 requests per second, continuously creating new threads increases CPU usage & memory consumption & making server slow.

ii)

Creating & destroying thousands of threads consumes significant CPU time.

Each thread requires its own stack memory, causing memory usage.

Frequent thread creation also increases context-switch overhead.

As a result server becomes slower & less efficient.

iii) Optimization : Use thread pool

Create a fixed number of worker threads & reuse them for incoming request.

Optimization 2: Limit number of threads

Set a maximum of active threads and use a request queue.

8) Bankers Algorithm:

(i) Need Matrix:

$$\text{Need} = \text{Maximum} - \text{Allocation}$$

Process	A	B	C
P ₁	1	1	1
P ₂	1	1	1
P ₃	2	1	1

(ii)

check if: request $\leq$ need & available

$$\text{Available} = (2, 1, 1) - (1, 1, 1) = (1, 0, 0)$$

$$P_2 \rightarrow (2, 2, 1)$$

$$P_1 \rightarrow (3, 2, 2)$$

P₃ finishes

Safe sequence: P₂ $\rightarrow$ P₁ $\rightarrow$ P₃

(iii)

There is no unsafe condition.

9) Deadlock Detection & Recovery:

(i)

Yes deadlock exists.

$T_1 \rightarrow R_2 \rightarrow T_2 \rightarrow R_3 \rightarrow T_3 \rightarrow R_4 \rightarrow T_4 \rightarrow R_1 \rightarrow T_1$

Since each transaction waiting for a resource held by another, none can proceed.

(ii)

It detects cycle in Resource Allocation graph. A cycle indicates all transactions are waiting and confirms deadlock.

(iii)

Terminate one of transactions to break cycle.

Preempt resources from selected transaction.

10) Monitor Based Synchronization.

(i) Monitor uses `wait()` when printer is busy & doesn't call `signal()` after printer becomes free. So processes waiting are never awakened.

(ii) After printing, busy is set to false, but no `signal()` is issued. Therefore waiting process remain blocked.

(iii)

monitor printer

{

boolean busy = False;

procedure print()

{

while (busy)

wait();

busy = true;

busy = false;

signal();

}

}